

Assignment-2: Path Tracer

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: Performance Analysis & Evaluation

May 27, 2026

Contents

1. Intro	3
2. Improvement 1: Lock Contention	3
2.1. Problem	3
2.2. Solution	3
3. Improvement 2: Inlining	5
3.1. Problem	5
3.2. Solution	5
4. Improvement 3: Thread Pooling	6
4.1. Problem	6
4.2. Solution	7
5. Improvement 4: Parallel Build	9
6. Improvement 5: Structure of Arrays & SIMD	10
7. Miscellaneous improvements	11
8. Benchmarking	12
9. Conclusion	12
10. Appendix	13
10.1. Platform	13
Bibliography	14

1. Intro

This report will discuss the improvements made to the CPU based pathtracer for Assignment-2 of the course: 'Performance Analysis & Evaluation'. Each major improvement will be discussed in their respective sections. A collection of miscellaneous improvements will be discussed in Section 7. Finally, the latest version of the implementation will be thoroughly discussed in Section 8.

2. Improvement 1: Lock Contention

This section will discuss the improvement of removing lock contention in the pathtracer implementation.

2.1. Problem

The first problem that was identified & solved is the lock contention in the `random.c` file. When first benchmarking the application, it became clear that increasing the thread count made the application significantly slower.

Initial sourcing for this became clear after using `perf record` & `perf lock`, for which the result can be seen in (TODO/ADD). This same behavior is visible in Figure 1, where the gray lines between the thread activity is the thread waiting for a lock to be finished.



Figure 1: AMD µProf - Lock Contention Threads

Further investigating the file, the offending function was identified to be `rand()`. Looking up the man page of the function [1], it becomes clear that the function is not thread safe and suffers from heavy lock contention in thread heavy workloads.

2.2. Solution

Included in the man page of the `rand()` function is a recommendation to use the safe function `rand_r()`, but this function seems to require enabling some posix standard and is deprecated.

The recommendation from online forum is a random value per thread using a thread local variable see in Listing 1.

```
1 static __thread uint32_t tls_seed = 0;
```

Listing 1:

In combination with `xorshift32` from [2] as shown in Listing 2.

```
1 static uint32_t xorshift32(void) {  
2     if (tls_seed == 0)  
3         tls_seed = (uint32_t)(uintptr_t)pthread_self() ^  
            (uint32_t)time(NULL);  
4     tls_seed ^= tls_seed << 13;  
5     tls_seed ^= tls_seed >> 17;  
6     tls_seed ^= tls_seed << 5;  
7     return tls_seed;  
8 }
```

Listing 2:

With the updated `random_double()` shown in Listing 3, there are some additional operations on the result and multiplication to further increase the randomness of the result.

```
1 double random_double(void) { return (xorshift32() >> 8) * (1.0 /  
    16777216.0); }
```

Listing 3:

3. Improvement 2: Inlining

This section will discuss the process of inlining [3] compute & time intensive operations to improve performance.

3.1. Problem

3.2. Solution

4. Improvement 3: Thread Pooling

This section will discuss the implementation of adding a thread pool which contains image rendering tasks. Each task will be responsible for a square tile of the image to be rendered.

4.1. Problem

Analyzing the application, using the AMD μ Prof [4] application, and viewing the thread section, shows the behavior visible in Figure 3.

Specifically the end of the timeline of the renderer is import. There is can be seen that the rendered splits the image in non equal work vertical slices as illustrated in Figure 2.

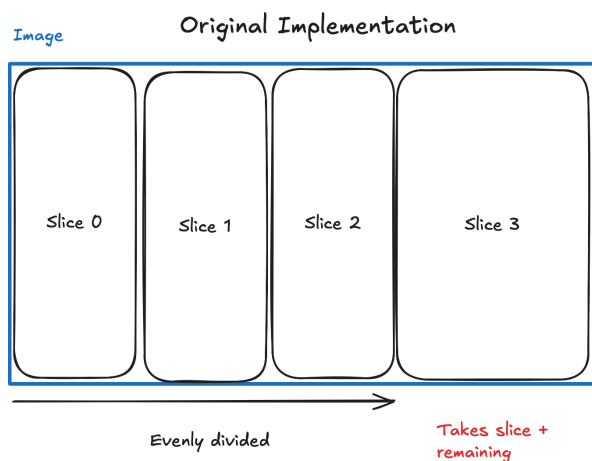


Figure 2: Renderer - Image Slicing Before

Depending on the image to be rendered, some vertical slices may have less work than other vertical slices. Additionally, the last vertical slice of the image is padded so it rendered the full with of the image.

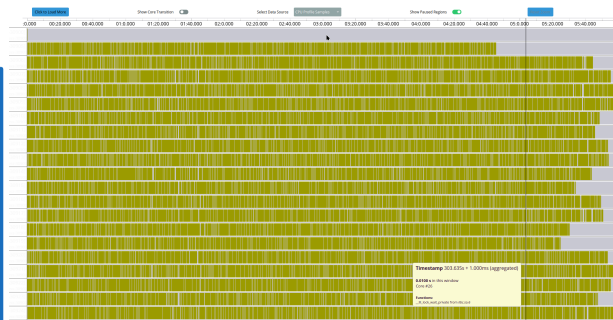


Figure 3: AMD μ Prof - Thread behavior

```

1 // renderer_render
2 for (int thread = 0; thread < renderer->threads; thread++)
3 {
4     thread_infos[thread].renderer = renderer;
5     thread_infos[thread].width_start = thread * width_per_thread;
6     if (thread == renderer->threads - 1)
7     {
8         // The last thread will take care of any remaining pixels if the
        width is not perfectly divisible by the number of threads.
9         thread_infos[thread].width_end = renderer->width;
10    }
11    else
12    {
13        thread_infos[thread].width_end = (thread + 1) * width_per_thread;
14    }
15
16    pthread_create(&threads[thread], NULL, (void (*)(void
        *))(renderer_render_part, &thread_infos[thread]));
17 }

```

Listing 4:

As is visible in Listing 4, the behavior mentioned earlier is visible. And the vertical slice behavior in Listing 5.

```

1 // renderer_render_part
2 for (int w = width_start; w < width_end; w++)
3 {
4     for (int h = 0; h < renderer->height; h++)
5     {
6         struct vec3 *color = &renderer->framebuffer[w + h * renderer-
        >width];
7
8         for (int sample = 0; sample < renderer->scene->samples; sample++)
9         {
10             render_pixel(renderer, color, w, h);
11         }
12     }
13 }

```

Listing 5:

4.2. Solution

The problem identified above was solved by two additions. First, instead of vertical slices, the image is now split in smaller (32x32) square tiles, as illustrated in Figure 4. In addition to the image tiling, render tasks are now based on pool design [5], [6].

The improvement in evenly shared work between the threads can be seen in Figure 5. The end of the timeline, the threads finish more at the same time and there are no more threads that are sitting idle.

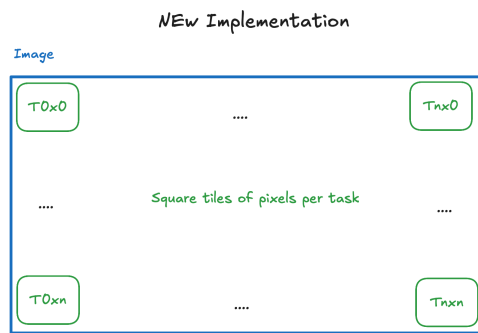


Figure 4: ...

4.2.1. Implementation

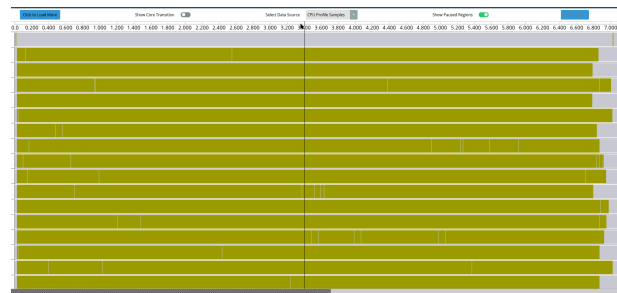


Figure 5: ...

5. Improvement 4: Parallel Build

6. Improvement 5: Structure of Arrays & SIMD

7. Miscellaneous improvements

- render pixel, 'lanes', store, multiple samples together
- post process pixels add the end and SIMD

8. Benchmarking

9. Conclusion

10. Appendix

10.1. Platform

The benchmarks were executed on a KUbuntu 25.04 desktop, with the specifications listed in Table 1.

PART	VALUE
CPU	Ryzen 9 5950X
RAM	64GB (3200 Mhz)
OS	TODO
Kernel Version	TODO
Scheduling	TODO
Make	TODO
GCC	TODO

Table 1: Desktop Specifications

Note: for the scheduling, all benchmark are executed using `taskset`. Due to `benchkit` limitation, the `taskset` `cpu` list is set to all available cores and the `thread` variable is varied.

Bibliography

- [1] “rand(3) — Linux manual page.” [Online]. Available: <https://www.man7.org/linux/man-pages/man3/rand.3.html>[◦]
- [2] Wikipedia, “Xorshift.” [Online]. Available: <https://en.wikipedia.org/wiki/Xorshift>[◦]
- [3] “Inlining.” [Online]. Available: https://en.wikipedia.org/wiki/Inline_expansion[◦]
- [4] AMD, “AMD μ Prof.” [Online]. Available: <https://www.amd.com/en/developer/uprof.html>[◦]
- [5] John, “Thread Pool in C.” [Online]. Available: <https://nachtimwald.com/2019/04/12/thread-pool-in-c/>[◦]
- [6] Unknown, “Creating a Thread Pool in C.” [Online]. Available: https://markc.su/posts/threadpool_c[◦]